

Smart Student Marks Organizer

Submitted By

Student Name	Student ID
Sk. Azmain Zunaeid	251-15-222
Md. Maruf Mia	251-15-042
Syeda Zawada Farah	251-15-223
Tahmid Ibne Mofazzol	251-15-548
Md. Delower Sarker	251-15-457

LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the Course

CSE214: Algorithms Lab

in the Department of Computer Science and Engineering



DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

19/04/2026

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Ms. Nishat Sadaf Lira, Lecturer**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

Ms. Nishat Sadaf Lira
Lecturer,
Department of Computer Science and Engineering
Daffodil International University

Submitted By

 Sk. Azmain Zunaid (Leader) ID: 251-15-222 Dept. of CSE, DIU	
 Md. Maruf Mia ID: 251-15-042 Dept. of CSE, DIU	 Syeda Zawada Farah ID: 251-15-223 Dept. of CSE, DIU
 Tahmid Ibne Mofazzol ID: 251-15-548 Dept. of CSE, DIU	 Md. Delower Sarker ID: 251-15-457 Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course has course outcomes as follows:

Table 1: Course Outcome Statements

CO's	Statements
CO1	Learn fundamental algorithms to imitate and analyze basic engineering problems.
CO2	Demonstrate skills to identify, analyze and formulate to choose appropriate algorithms to address engineering problems.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	LD	KP	CEP	CEA
CO1	PO1	C1	K2	EP1	-
CO2	PO2	C3	K2, K3	EP3	-

The mapping justification of this table is provided in the sections 4.3.1, and 4.3.2.

Table of Contents

Declaration.....	i
Course & Program Outcome.....	ii
1 Introduction.....	1
1.1 Introduction.....	1
1.2 Motivation.....	1
1.3 Objectives.....	2
1.4 Feasibility Study.....	2
1.5 Gap Analysis.....	2
1.6 Project Outcome.....	3
2 System Architecture.....	4
2.1 Requirement Analysis & Design Specification.....	4
2.1.1 Requirements.....	4
2.1.2 Syestem Architecture.....	5
2.2 Proposed Methodology/System Design.....	5
2.2.1 UI Design.....	6
2.2.2 Overall Project Plan.....	6
3 Implementation and Results.....	8
3.1 Implementation.....	8
3.2 Time Complexity Analysis.....	21
3.3 Space Complexity Analysis.....	22
3.4 Results and Discussion.....	22
3.5 Sample Input-Output.....	23
4 Engineering Standards and Mapping.....	28
4.1 Impact on Society, Environment and Sustainability.....	28
4.1.1 Impact on Life.....	28
4.1.2 Impact on Society & Environment.....	28
4.2 Project Management and Team Work.....	29
4.3 Complex Engineering Problem.....	30
4.3.1 Mapping of Program Outcome.....	30
4.3.2 Complex Problem Solving.....	30
5 Conclusion.....	31
5.1 Summary.....	31
5.2 Limitations.....	31
5.3 Future Work.....	32
References.....	33

Chapter 1

Introduction

This chapter provides a comprehensive overview of the **Smart Student Marks Organizer**, detailing its purpose, motivational factors, specific objectives, feasibility considerations, identified gaps in existing solutions, and expected project outcomes.

1.1 Introduction

Smart Student Marks Organizer is a console-based application developed in ANSI C that implements a comprehensive **student record management system**. The system provides functionality to add student records, view top-performing students using Quick Sort, explore students by section, generate section-wise bar graphs for course performance, search students using Binary Search, and remove student records.

The system maintains student records using a **singly linked list** data structure with **persistent storage** in `cse_students_db.txt`. Each student node contains: unique ID, name, section, marks for five courses (Algorithms, Algorithms Lab, Engineering Mathematics, Discrete Mathematics, Bangladesh Studies), and total marks calculated automatically. The linked list is maintained in **sorted order** by **student ID** during insertion, ensuring **efficient searching** operations.

The system demonstrates two fundamental algorithms:

1. **Quick Sort Algorithm** — Used to sort students by total marks in descending order to identify the top 3 performers (Hall of Fame)
2. **Binary Search Algorithm** — Used to efficiently search for a student by ID in $O(\log n)$ time

This project effectively demonstrates the practical application of algorithm design principles while addressing real-world academic record management challenges. It serves as both a functional prototype and an educational tool, illustrating core computer science concepts—such as divide-and-conquer (Quick Sort), efficient searching (Binary Search), dynamic memory management, pointer manipulation, and file I/O operations—within an accessible, real-world context.

1.2 Motivation

The development of this system was motivated by two primary factors:

Pedagogical Motivation: Algorithms like Quick Sort and Binary Search often remain abstract concepts for computer science students. This project bridges the gap between theoretical knowledge and practical application by implementing these algorithms in a realistic academic scenario. Students can observe how divide-and-conquer strategy works in Quick Sort and how logarithmic-time search works in Binary Search, enhancing comprehension and retention.

Practical Motivation: Educational institutions require efficient student record management systems that can organize data, generate rankings, and provide quick search capabilities. By developing a console-based solution in C, the project demonstrates how fundamental algorithmic concepts can solve real-world problems without the overhead of complex frameworks.

1.3 Objectives

The project aims to achieve the following specific objectives:

1. Implement persistent student record management using linked lists with file-based storage
2. Apply Quick Sort algorithm to identify top-performing students (Top 3 ranking)
3. Implement Binary Search algorithm for efficient student lookup by ID
4. Create section-wise analysis tools including bar graph visualization of course averages
5. Maintain linked list in sorted order (by ID) during insertion for optimal search performance

1.4 Feasibility Study

The feasibility of Smart Student Marks Organizer was assessed through multiple dimensions:

Technical Feasibility: The project utilizes standard C libraries (stdio.h, stdlib.h, string.h, ctype.h) without external dependencies, making it highly portable and compatible with any ANSI C compliant compiler. The console-based interface eliminates complex GUI requirements.

Resource Feasibility: Minimal hardware requirements (1GB RAM, standard processor) ensure accessibility. The text-based storage system requires negligible disk space while maintaining data persistence.

Operational Feasibility: The menu-driven system provides intuitive navigation. Input validation and error handling mechanisms ensure robust operation even with invalid user inputs.

Algorithmic Feasibility: Quick Sort provides $O(n \log n)$ average-case time complexity for sorting, making it efficient for ranking operations. Binary Search provides $O(\log n)$ search time, significantly faster than linear search for large datasets.

1.5 Gap Analysis

Current educational tools and academic management systems exhibit several gaps that this project addresses:

Identified Gaps in Prior Projects: Earlier student record systems often used arrays with fixed size limits or linear search for student lookup. They lacked algorithmic efficiency for ranking and searching operations. This project overcomes these limitations by implementing:

- Dynamic linked list storage (no fixed size limits)
- Quick Sort for efficient ranking ($O(n \log n)$)
- Binary Search for fast student lookup ($O(\log n)$)
- Section-wise performance analysis with bar graph visualization

Educational Tools Gap: Most algorithm visualizers focus on abstract animations but neglect practical aspects like file persistence and real-world constraints. Smart Student Marks Organizer integrates both theoretical algorithms and practical implementation challenges.

Commercial Systems Gap: Existing academic management systems are often feature-heavy and implemented in high-level languages, obscuring the fundamental algorithm operations. This project maintains transparency in demonstrating core algorithmic concepts.

1.6 Project Outcome

The project delivers several significant outcomes:

1. Working student record system with automatic saving to/loading from `cse_students_db.txt`
2. Quick Sort implementation that correctly identifies top 3 students by total marks
3. Binary Search implementation for $O(\log n)$ student lookup by ID
4. Section-wise analysis tools including bar graph generation for course averages
5. Educational prototype showing linked lists, Quick Sort, Binary Search, memory management, and file I/O in a tangible academic context

Chapter 2

System Architecture

This chapter details the system requirements, architectural design, user interface specifications, and project planning methodology employed in the development of Smart Student Marks Organizer.

2.1 Requirement Analysis & Design Specification

2.1.1 Requirements

Hardware Requirements:

- Processor: 1 GHz or higher
- Memory: 1 GB RAM minimum
- Storage: 10 MB available space
- Input: Standard keyboard
- Output: Console/terminal display

Software Requirements:

- Operating System: Windows/Linux/macOS
- Compiler: GCC, Clang, or any ANSI C compliant compiler
- Dependencies: Standard C libraries only (no external dependencies)

Functional Requirements:

- **Add Student Record:** Insert new student with ID, name, section, and 5 course marks; maintain sorted order by ID
- **Top 3 Ranking:** Apply Quick Sort to display highest-performing students
- **Explore by Section:** Filter and display all students in a specific section
- **Bar Graph Generation:** Calculate course averages and display visual representation
- **Find Student:** Implement Binary Search for efficient student lookup by ID
- **Remove Student:** Delete student record from database
- **Persistence:** Automatic saving and loading of student data

Non-Functional Requirements:

- **Performance:** Quick Sort $O(n \log n)$, Binary Search $O(\log n)$
- **Reliability:** Consistent operation without crashes
- **Usability:** Intuitive menu-driven interface
- **Maintainability:** Well-documented, modular code structure
- **Memory Safety:** Proper allocation and deallocation of dynamic memory

2.1.2 System Architecture

The system architecture comprises **three** core modules:

Student Record Management Module: Handles all student operations using a singly linked list structure. Each node contains student ID, name, section, five course marks, total marks, and next pointer. The list is maintained in ascending order by student ID for optimal Binary Search performance.

Algorithm Module: Implements two key algorithms:

- **Quick Sort:** Recursive divide-and-conquer algorithm for sorting students by total marks in descending order. Uses Hoare partition scheme with pivot selection as first element.
- **Binary Search:** Iterative logarithmic-time search algorithm for finding students by ID in the sorted linked list (converted to array for efficient random access).

File I/O Module: Handles data persistence through `cse_students_db.txt` using semicolon-delimited formatting for reliable storage and retrieval.

2.2 Proposed Methodology/System Design

The system follows a structured design approach:

Data Flow:

1. Program starts → `loadData()` reads existing records from file
2. Main menu presents 7 options to user
3. Each operation manipulates the linked list and calls `saveData()` for persistence
4. Quick Sort operates on array of pointers to avoid modifying original linked list
5. Binary Search operates on sorted array of student pointers

Algorithm Selection Justification:

Algorithm	Use Case	Time Complexity	Why Chosen
Quick Sort	Top 3 Ranking	$O(n \log n)$ average	Fast in-place sorting, excellent for academic ranking
Binary Search	Find Student	$O(\log n)$	Logarithmic search, ideal for ID-based lookup
Linked List	Data Storage	$O(n)$ insertion/search	Dynamic size, easy insertion/deletion

2.2.1 UI Design

The user interface employs a console-based hierarchical menu system:

Main Menu:

```
=====
                        SMART STUDENT MARKS ORGANIZER
=====
[1] Add Student Record
[2] Top 3 Ranking (CSE Dept.)
[3] Explore by Section
[4] View Section Wise Courses (Bar Graph)
[5] Find Student
[6] Remove Student
[7] View All Students Ranked
[0] Exit
=====
Select action: █
```

UI Characteristics:

- Consistent formatting and clear section headers
- Informative prompts and error messages
- Responsive feedback for all user actions
- Input validation with helpful error guidance (name validation: letters/spaces only, marks: 0-100 range)

2.2.2 Overall Project Plan

The project followed a structured development timeline:

Phase 1: Requirements & Design (1 Week)

- Requirement analysis and specification
- Data structure design (Student struct, linked list)
- Algorithm selection (Quick Sort, Binary Search)

Phase 2: Core Implementation (3 Weeks)

- Linked list implementation with sorted insertion
- File I/O operations for data persistence
- Quick Sort algorithm implementation
- Binary Search algorithm implementation

Phase 3: Validation & Enhancement (2 Weeks)

- Comprehensive input validation
- Error handling mechanisms
- Memory management optimization
- Edge case handling (empty database, single student, etc.)

Phase 4: Testing & Debugging (1 Week)

- Unit testing of individual modules
- Algorithm correctness testing
- Integration testing
- Performance optimization

Phase 5: Documentation & Finalization (2 Weeks)

- Code documentation and commenting
- Report preparation
- Presentation development
- Final submission preparation

Overall Project Plan:

Weeks	Phase
Week 1	P-1: Requirements & Design
Week 2-4	P-2: Core Implementation
Week 5-6	P-3: Validation & Enhancement
Week 7	P-4: Testing & Debugging
Week 8-9	P-5: Documentation & Finalization

Chapter 3

Implementation and Results

This chapter provides detailed insights into the system implementation, algorithm analysis, performance characteristics, and operational results of the Smart Student Marks Organizer.

3.1 Implementation

The system was implemented as a single-file C program following ANSI C standards. The implementation emphasizes modularity, algorithm correctness, memory safety, and robust error handling.

Development Environment:

- **Compiler:** GCC (GNU Compiler Collection)
- **IDE:** Visual Studio Code with C/C++ extension
- **Debugger:** GDB for debugging sessions
- **Platform:** Cross-platform (Windows/Linux/macOS compatible)

Source Code:

```
1  /* ##### SSMO CSE214 Project Binary Brains68_A1 DIU ##### */
2  /* ----- Libraries(4) - Both----- */
3
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <string.h>
7  #include <ctype.h>
8
9  /* ----- Structure (1) ----- */
10
11 struct Student
12 {
13     int id;
14     char name[50];
15     char section[15];
16     int courses[5];
17     int total_marks;
18     struct Student *next;
19 };
20
21 struct Student *start = NULL;
22
```

```

23  /* ----- File Operations (2) ----- */
24
25  void saveData() // For saving all the operational data permanently
26  {
27      FILE *fp = fopen("cse_students_db.txt", "w");
28      if(fp == NULL) return;
29
30      struct Student *i = start;
31      while(i != NULL)
32      {
33          fprintf(fp, "%d;%s;%s;%d;%d;%d;%d;%d;%d\n",
34                  i->id, i->name, i->section,
35                  i->courses[0], i->courses[1], i->courses[2],
36                  i->courses[3], i->courses[4], i->total_marks);
37          i = i->next;
38      }
39      fclose(fp);
40  }
41

```

```

42  void loadData() // For loading all the previous operational data just after starting the program
43  {
44      FILE *fp = fopen("cse_students_db.txt", "r");
45      if(fp == NULL) return;
46
47      char line[200];
48      while(fgets(line, sizeof(line), fp))
49      {
50          int id, c1, c2, c3, c4, c5, total;
51          char name[50], section[15];
52
53          if(sscanf(line, "%d;%49[^\n];%14[^\n];%d;%d;%d;%d;%d;%d", &id, name, section, &c1, &c2, &c3, &c4, &c5, &total) == 9)
54          {
55              struct Student *newnode = (struct Student*)malloc(sizeof(struct Student));
56              newnode->id = id;
57              strcpy(newnode->name, name);
58              strcpy(newnode->section, section);
59              newnode->courses[0] = c1;
60              newnode->courses[1] = c2;
61              newnode->courses[2] = c3;
62              newnode->courses[3] = c4;
63              newnode->courses[4] = c5;
64              newnode->total_marks = total;
65              newnode->next = NULL;
66
67              if(start == NULL) start = newnode;
68              else
69              {
70                  struct Student *t = start;
71                  while(t->next != NULL) t = t->next;
72                  t->next = newnode;
73              }
74          }
75      }
76      fclose(fp);
77  }
78

```

```

79 | /* ##### SINGLY LINKED LIST & CORE OPERATIONS (3) ##### */
80 |
81 | /* ----- Insert Function (Sorted with Validation) (1) ----- */
82 |
83 | void addStudentSorted()
84 | {
85 |     struct Student *newnode = (struct Student*)malloc(sizeof(struct Student));
86 |     struct Student *temp;
87 |     int exists, valid;
88 |
89 |     printf("\n--- [ ADD NEW CSE STUDENT ] ---\n");
90 |
91 |     // STEP 1: ID Validation
92 |     while(1)
93 |     {
94 |         printf("Enter Student ID: ");
95 |         if(scanf("%d", &newnode->id) != 1)
96 |         {
97 |             while(getchar()!='\n');
98 |             printf("Invalid input! Please enter a number.\n");
99 |             continue;
100 |        }
101 |
102 |        temp = start;
103 |        exists = 0;
104 |        while(temp != NULL)
105 |        {
106 |            if(temp->id == newnode->id)
107 |            {
108 |                exists = 1;
109 |                break;
110 |            }
111 |            temp = temp->next;
112 |        }
113 |
114 |        if(exists) printf("Error: This ID already exists! Please enter a unique ID.\n");
115 |        else break;
116 |    }
117 |
118 |    // STEP 2: Name Validation (Alphabets only)
119 |    while(1)
120 |    {
121 |        printf("Enter Student Name: ");
122 |        scanf(" %[^\\n]", newnode->name);
123 |
124 |        valid = 1;
125 |        for(int i = 0; newnode->name[i] != '\\0'; i++)
126 |        {
127 |            // Allow alphabets, spaces, and standard dots (like Md.)
128 |            if(!isalpha(newnode->name[i]) && !isspace(newnode->name[i]) && newnode->name[i] != '.')
129 |            {
130 |                valid = 0;
131 |                break;
132 |            }
133 |        }
134 |
135 |        if(!valid) printf("Error: Name must contain only letters and spaces (Ex: Mohammad).\n");
136 |        else break;
137 |    }
138 |
139 |    // STEP 3: Section Input
140 |    printf("Enter Section (Ex: 68_A, 68_B): ");
141 |    scanf("%14s", newnode->section);
142 |

```

```

143 // STEP 4: Marks Validation (0 to 100)
144 const char *course_names[] =
145 {
146     "Algorithms(CSE213)", "Algorithms Lab(CSE214)", "Engineering Mathematics(MAT211)",
147     "Discrete Mathematics(CSE212)", "Bangladesh Studies(BNS101)"
148 };
149
150 printf("\n-- Enter Course Marks (0 - 100) --\n");
151 for(int c = 0; c < 5; c++)
152 {
153     while(1)
154     {
155         printf("%s: ", course_names[c]);
156         if(scanf("%d", &newnode->courses[c]) != 1)
157         {
158             while(getchar() != '\n');
159             printf("Invalid input! Please enter a number.\n");
160             continue;
161         }
162
163         // Checking the 0 to 100 limit
164         if(newnode->courses[c] < 0 || newnode->courses[c] > 100)
165         {
166             printf("Error: Marks must be between 0 and 100. Please try again.\n");
167         }
168         else break;
169     }
170 }
171
172 newnode->total_marks = newnode->courses[0] + newnode->courses[1] + newnode->courses[2] + newnode->courses[3] + newnode->courses[4];
173
174
175 newnode->next = NULL;
176
177 // STEP 5: Sorted Insertion Logic (By ID)
178 if (start == NULL || start->id >= newnode->id)
179 {
180     newnode->next = start;
181     start = newnode;
182 }
183 else
184 {
185     struct Student *current = start;
186     while (current->next != NULL && current->next->id < newnode->id)
187     {
188         current = current->next;
189     }
190     newnode->next = current->next;
191     current->next = newnode;
192 }
193
194 printf("\n Student added successfully!\n");
195
196
197 saveData();
198 }
199

```



```

200 | /* ----- Section Explorer (2) ----- */
201 |
202 | void exploreBySection()
203 | {
204 |     char sec[15];
205 |     printf("\n--- [ CSE SECTION EXPLORER ] ---\n");
206 |     printf("Enter Section to view ( Ex: 68_A): ");
207 |     scanf("%14s", sec);
208 |
209 |     printf("\nFetching data for Section %s...\n", sec);
210 |     printf("--- [ SECTION %s: COURSE WISE LIST ] ---\n", sec);
211 |
212 |     printf("%-6s | %-18s | %-3s | %-3s | %-3s | %-3s | %-3s | %-5s\n", "ID", "Name", "Alg", "Alg_L", "EM", "DM", "BNS", "Total");
213 |     printf("-----\n");
214 |
215 |     struct Student *i = start;
216 |     int found = 0;
217 |     while(i != NULL)
218 |     {
219 |         if(strcmp(i->section, sec) == 0)
220 |         {
221 |             printf("%-6d | %-18s | %-3d | %-5d | %-3d | %-3d | %-3d | %-5d\n",
222 |                 i->id, i->name, i->courses[0], i->courses[1], i->courses[2], i->courses[3], i->courses[4], i->total_marks);
223 |             found = 1;
224 |         }
225 |         i = i->next;
226 |     }
227 |
228 |     if(!found) printf("No students found in Section %s.\n", sec);
229 |     printf("-----\n");
230 | }
231 |

```



```

232 | /* ----- Remove Student Function (3) ----- */
233 |
234 | void removeStudentByID()
235 | {
236 |     int target_id;
237 |     printf("\n--- [ REMOVE STUDENT RECORD ] ---\n");
238 |     printf("Enter Student ID to remove: ");
239 |
240 |     if(scanf("%d", &target_id) != 1)
241 |     {
242 |         while(getchar()!='\n');
243 |         printf("Invalid input! Please enter a valid ID number.\n");
244 |         return;
245 |     }
246 |
247 |     if(start == NULL)
248 |     {
249 |         printf("\nDatabase is empty. No students to remove.\n");
250 |         return;
251 |     }
252 |
253 |     struct Student *temp = start;
254 |     struct Student *prev = NULL;
255 |
256 |     if (temp != NULL && temp->id == target_id)
257 |     {
258 |         start = temp->next;
259 |         free(temp);
260 |         printf("\n Student with ID %d has been removed successfully!\n", target_id);
261 |         saveData();
262 |         return;
263 |     }
264 |
265 |     while (temp != NULL && temp->id != target_id)
266 |     {
267 |         prev = temp;
268 |         temp = temp->next;
269 |     }
270 |
271 |     if (temp == NULL)
272 |     {
273 |         printf("\n Student with ID %d not found in the database.\n", target_id);
274 |         return;
275 |     }
276 |
277 |     prev->next = temp->next;
278 |     free(temp);
279 |
280 |     printf("\n Student with ID %d has been removed successfully!\n", target_id);
281 |     saveData();
282 | }
283 |

```

```

284  /* ##### ALGORITHMS (QUICK SORT & BINARY SEARCH) (5) ##### */
285
286  int countStudents() // For count all the student
287  {
288      int count = 0;
289      struct Student *i = start;
290      while(i != NULL)
291      {
292          count++;
293          i = i->next;
294      }
295      return count;
296  }
297
298  /* ----- Quick Sort Implementation ----- */
299
300  void swap(struct Student **a, struct Student **b)
301  {
302      struct Student *temp = *a;
303      *a = *b;
304      *b = temp;
305  }
306
307  int partition(struct Student *arr[], int low, int high)
308  {
309      int pivot = arr[low]->total_marks;
310      int i = low;
311      int j = high;
312
313      while (i < j)
314      {
315          while (arr[i]->total_marks >= pivot && i <= high - 1) i++;
316          while (arr[j]->total_marks < pivot && j >= low + 1) j--;
317          if (i < j) swap(&arr[i], &arr[j]);
318      }
319      swap(&arr[low], &arr[j]);
320      return j;
321  }
322
323  void quickSort(struct Student *arr[], int low, int high)
324  {
325      if (low < high)
326      {
327          int pivot_idx = partition(arr, low, high);
328          quickSort(arr, low, pivot_idx - 1);
329          quickSort(arr, pivot_idx + 1, high);
330      }
331  }
332

```

```

333 | void viewTopThree() // View Top 3 Ranking
334 | {
335 |     int n = countStudents();
336 |     if(n == 0)
337 |     {
338 |         printf("\nNo student records available.\n");
339 |         return;
340 |     }
341 |
342 |     struct Student **arr = malloc(n * sizeof(struct Student*));
343 |     struct Student *curr = start;
344 |     for(int i = 0; i < n; i++)
345 |     {
346 |         arr[i] = curr;
347 |         curr = curr->next;
348 |     }
349 |
350 |     printf("\n--- [ HALL OF FAME: TOP 3 CSE STUDENTS ] ---\n");
351 |
352 |     quickSort(arr, 0, n - 1);
353 |
354 |     printf("%-5s | %-6s | %-20s | %-7s | %-5s\n", "RANK", "ID", "NAME", "SECTION", "TOTAL");
355 |     printf("-----\n");
356 |
357 |     int limit = (n < 3) ? n : 3;
358 |     for(int i = 0; i < limit; i++)
359 |     {
360 |         printf(" #%-3d | %-6d | %-20s | %-7s | %-5d\n",
361 |             i + 1, arr[i]->id, arr[i]->name, arr[i]->section, arr[i]->total_marks);
362 |     }
363 |     printf("-----\n");
364 |     free(arr);
365 | }
366 |

```

```

367 | /* ----- View All Students Ranked ----- */
368
369 void viewAllSorted()
370 {
371     int n = countStudents();
372     if(n == 0)
373     {
374         printf("\nNo student records available.\n");
375         return;
376     }
377
378     struct Student **arr = malloc(n * sizeof(struct Student*));
379     struct Student *curr = start;
380     for(int i = 0; i < n; i++)
381     {
382         arr[i] = curr;
383         curr = curr->next;
384     }
385
386     printf("\n--- [ FULL RANKING: ALL CSE STUDENTS ] ---\n");
387
388     quickSort(arr, 0, n - 1);
389
390     printf("%-5s | %-6s | %-20s | %-7s | %-5s\n", "RANK", "ID", "NAME", "SECTION", "TOTAL");
391     printf("-----\n");
392
393     for(int i = 0; i < n; i++)
394     {
395         printf(" #%-3d | %-6d | %-20s | %-7s | %-5d\n",
396             i + 1, arr[i]->id, arr[i]->name, arr[i]->section, arr[i]->total_marks);
397     }
398     printf("-----\n");
399     free(arr);
400 }
401
402 | /* ----- Binary Search Student Finder ----- */
403
404 int binarySearch(struct Student *arr[], int low, int high, int target)
405 {
406     while (low <= high)
407     {
408         int mid = low + (high - low) / 2;
409         if (arr[mid]->id == target) return mid;
410         if (arr[mid]->id < target) low = mid + 1;
411         else high = mid - 1;
412     }
413     return -1;
414 }

```



```

415
416 void searchStudentByID() // Search by student ID
417 {
418     int target_id;
419     printf("\n--- [ SEARCH STUDENT ] ---\n");
420     printf("Enter Student ID to search: ");
421     scanf("%d", &target_id);
422
423     int n = countStudents();
424     if(n == 0)
425     {
426         printf("\nDatabase is empty.\n");
427         return;
428     }
429
430     struct Student **arr = malloc(n * sizeof(struct Student*));
431     struct Student *curr = start;
432     for(int i = 0; i < n; i++)
433     {
434         arr[i] = curr;
435         curr = curr->next;
436     }
437
438     int index = binarySearch(arr, 0, n - 1, target_id);
439
440     // Shows all courses besides total marks
441     if(index != -1)
442     {
443         printf("\n--- [ STUDENT RECORD FOUND ] ---\n");
444         printf("ID      : %d\n", arr[index]->id);
445         printf("Name    : %s\n", arr[index]->name);
446         printf("Section : %s\n", arr[index]->section);
447         printf("-----\n");
448         printf("Algorithms (CSE213)      : %d\n", arr[index]->courses[0]);
449         printf("Algorithms Lab (CSE214)  : %d\n", arr[index]->courses[1]);
450         printf("Engineering Math (MAT211) : %d\n", arr[index]->courses[2]);
451         printf("Discrete Math (CSE212)   : %d\n", arr[index]->courses[3]);
452         printf("Bangladesh Studies (BNS101): %d\n", arr[index]->courses[4]);
453         printf("-----\n");
454         printf("TOTAL MARKS              : %d\n", arr[index]->total_marks);
455         printf("-----\n");
456     }
457     else
458     {
459         printf("\nStudent not found!\n");
460     }
461
462     free(arr);
463 }
464

```

```

465  /* ----- Section Average Bar Graph ----- */
466
467  void generateSectionBarGraph()
468  {
469      char sec[15];
470      printf("\n--- [ SECTION AVERAGE BAR GRAPH ] ---\n");
471      printf("Enter Section to analyze (Ex: 68_A): ");
472      scanf("%14s", sec);
473
474      int sum[5] = {0, 0, 0, 0, 0};
475      int count = 0;
476
477      struct Student *i = start;
478      while(i != NULL)
479      {
480          if(strcmp(i->section, sec) == 0)
481          {
482              for(int c = 0; c < 5; c++)
483              {
484                  sum[c] += i->courses[c];
485              }
486              count++;
487          }
488          i = i->next;
489      }
490
491      if(count == 0)
492      {
493          printf("\nNo students found in Section %s to generate a graph.\n", sec);
494          return;
495      }
496
497      printf("\nGenerating Bar Graph for Section %s (Total Students: %d",
498
499      const char *course_names[] =
500      {
501          "Algorithms(CSE213)           ",
502          "Algorithms Lab(CSE214)          ",
503          "Engineering Mathematics(MAT211)   ",
504          "Discrete Mathematics(CSE212)      ",
505          "Bangladesh Studies(BNS101)       "
506      };
507
508      for(int c = 0; c < 5; c++)
509      {
510          float avg = (float)sum[c] / count;
511          printf("%s [Avg: %5.1f] : ", course_names[c], avg);
512
513          int bars = (int)(avg / 2);
514          for(int b = 0; b < bars; b++)
515          {
516              printf("#");
517          }
518          printf("\n");
519      }
520  }
521

```

```

522  /* ##### MAIN MENU (1) ##### */
523
524  int main()
525  {
526      loadData();
527      int choice;
528
529      while(1)
530      {
531          printf("\n===== \n");
532          printf("      SMART STUDENT MARKS ORGANIZER\n");
533          printf("===== \n");
534          printf(" [1] Add Student Record\n");
535          printf(" [2] Top 3 Ranking (CSE Dept.)\n");
536          printf(" [3] Explore by Section\n");
537          printf(" [4] View Section Wise Courses (Bar Graph)\n");
538          printf(" [5] Find Student \n");
539          printf(" [6] Remove Student\n");
540          printf(" [7] View All Students Ranked\n");
541          printf(" [0] Exit\n");
542          printf("===== \n");
543          printf(" Select action: ");
544
545          if(scanf("%d", &choice) != 1)
546          {
547              while(getchar() != '\n');
548              continue;
549          }

```

```

550
551     switch(choice)
552     {
553     case 1:
554         addStudentSorted();
555         break;
556     case 2:
557         viewTopThree();
558         break;
559     case 3:
560         exploreBySection();
561         break;
562     case 4:
563         generateSectionBarGraph();
564         break;
565     case 5:
566         searchStudentByID();
567         break;
568     case 6:
569         removeStudentByID();
570         break;
571     case 7:
572         viewAllSorted();
573         break;
574     case 0:
575         saveData();
576         printf("\n Exiting Smart Student Marks Organizer...\n");
577         exit(0);
578     default:
579         printf("Invalid Option. Please try again.\n");
580         break;
581     }
582 }
583 return 0;
584 }

```


Algorithm Explanation:

Quick Sort follows divide-and-conquer strategy. The partition function selects the first element as pivot and rearranges the array so elements greater than or equal to pivot come before elements less than pivot. The function returns the final pivot position. Quick Sort recursively applies partition to left and right subarrays, resulting in a fully sorted descending-order array.

Binary Search works on a sorted array by repeatedly dividing the search interval in half. If the target ID equals the middle element, the search terminates. If the target is greater, search continues in the upper half; otherwise, search continues in the lower half. This provides $O(\log n)$ search time.

3.2 Time Complexity Analysis

Function	Algorithm	Time Complexity	Space Complexity
addStudentSorted()	Sorted Linked List Insert	$O(n)$	$O(1)$
viewTopThree()	Quick Sort + Traversal	$O(n \log n)$	$O(n)$
binarySearch()	Binary Search	$O(\log n)$	$O(1)$
exploreBySection()	Linear Traversal	$O(n)$	$O(1)$
generateSectionBarGraph()	Linear Traversal	$O(n)$	$O(1)$
removeStudentByID()	Linear Search + Deletion	$O(n)$	$O(1)$
countStudents()	Linear Traversal	$O(n)$	$O(1)$
saveData()	File Write Traversal	$O(n)$	$O(1)$
loadData()	File Read + List Build	$O(n)$	$O(n)$

Justification:

- **Quick Sort:** Average case $O(n \log n)$, worst case $O(n^2)$ — for ranking operation
- **Binary Search:** $O(\log n)$ — significantly faster than linear $O(n)$ for large datasets
- **Linked List Operations:** $O(n)$ for traversal-based operations

3.3 Space Complexity Analysis

Component	Space Complexity	Explanation
Linked List Storage	$O(n)$	n = number of students
Quick Sort Array	$O(n)$	Temporary array of pointers
Binary Search Array	$O(n)$	Temporary array of pointers
File Storage	$O(n)$	Persistent storage
Recursive Quick Sort Stack	$O(\log n)$	Depth of recursion

3.4 Results and Discussion

Functional Testing Results

Add Student Record Testing:

- ✓ ID uniqueness validation working correctly
- ✓ Name validation (letters/spaces only) working
- ✓ Marks validation (0-100 range) working
- ✓ Sorted insertion maintaining ascending ID order
- ✓ Data persistence across sessions

Quick Sort / Top 3 Ranking Testing:

- ✓ Correctly identifies top 3 students by total marks
- ✓ Handles edge cases (0, 1, 2, 3+ students)
- ✓ Quick Sort implementation produces correct descending order

Binary Search Testing:

- ✓ Successfully finds existing student by ID
- ✓ Returns -1 for non-existent ID
- ✓ $O(\log n)$ performance verified

Section Explorer Testing:

- ✓ Correctly filters students by section
- ✓ Displays all five course marks and total
- ✓ Handles invalid sections gracefully

Bar Graph Generation Testing:

- ✓ Calculates correct course averages
- ✓ Visual representation proportional to average
- ✓ Handles empty sections with error message

Remove Student Testing:

- ✓ Deletes from beginning, middle, end of list
- ✓ Proper memory deallocation confirmed
- ✓ Updated list saved to file

Performance Validation

Execution Speed:

- Quick Sort on 100 students: < 0.001 seconds
- Binary Search on 1000 students: ~10 comparisons
- Linear operations: Instantaneous for typical class sizes ($n < 500$)

Memory Usage:

- Stable memory footprint during extended operation
- Proper cleanup after student removal
- No observable memory leaks in testing

3.5 Sample Input-Output

Main Menu

```
=====
      SMART STUDENT MARKS ORGANIZER
=====
[1] Add Student Record
[2] Top 3 Ranking (CSE Dept.)
[3] Explore by Section
[4] View Section Wise Courses (Bar Graph)
[5] Find Student
[6] Remove Student
[7] View All Students Ranked
[0] Exit
=====
select action: █
```

Add Student Record

```
=====
SMART STUDENT MARKS ORGANIZER
=====
[1] Add Student Record
[2] Top 3 Ranking (CSE Dept.)
[3] Explore by Section
[4] View Section Wise Courses (Bar Graph)
[5] Find Student
[6] Remove Student
[7] View All Students Ranked
[0] Exit
=====
Select action: 1

--- [ ADD NEW CSE STUDENT ] ---
Enter Student ID: 472
Enter Student Name: Arif
Enter Section (Ex: 68_A, 68_B): 68_A

-- Enter Course Marks (0 - 100) --
Algorithms(CSE213): 87
Algorithms Lab(CSE214): 78
Engineering Mathematics(MAT211): 85
Discrete Mathematics(CSE212): 79
Bangladesh Studies(BNS101): 87

Student added successfully!
```

Top 3 Ranking (68 Batch CSE Dept.)

```
=====
SMART STUDENT MARKS ORGANIZER
=====
[1] Add Student Record
[2] Top 3 Ranking (CSE Dept.)
[3] Explore by Section
[4] View Section Wise Courses (Bar Graph)
[5] Find Student
[6] Remove Student
[7] View All Students Ranked
[0] Exit
=====
Select action: 2

--- [ HALL OF FAME: TOP 3 CSE STUDENTS ] ---
RANK | ID | NAME | SECTION | TOTAL
-----
#1 | 450 | Mohammad | 68_B | 467
#2 | 222 | Azmain | 68_A | 429
#3 | 457 | Delower | 68_A | 424
-----
```

Explore by Section

```
=====
SMART STUDENT MARKS ORGANIZER
=====

[1] Add Student Record
[2] Top 3 Ranking (CSE Dept.)
[3] Explore by Section
[4] View Section Wise Courses (Bar Graph)
[5] Find Student
[6] Remove Student
[7] View All Students Ranked
[0] Exit

=====
Select action: 3

--- [ CSE SECTION EXPLORER ] ---
Enter Section to view ( Ex: 68_A): 68_A

Fetching data for Section 68_A...
--- [ SECTION 68_A: COURSE WISE LIST ] ---
ID      | Name           | Alg | Alg_L | EM | DM | BNS | Total
-----|-----|-----|-----|-----|-----|-----|-----
222     | Azmain         | 89  | 78    | 76 | 89 | 97  | 429
457     | Delower       | 89  | 90    | 78 | 78 | 89  | 424
472     | Arif           | 87  | 78    | 85 | 79 | 87  | 416
548     | Tahmid        | 98  | 87    | 76 | 78 | 78  | 417
-----|-----|-----|-----|-----|-----|-----|-----
```

View section wise courses (Bar Graph)

```
=====
SMART STUDENT MARKS ORGANIZER
=====

[1] Add Student Record
[2] Top 3 Ranking (CSE Dept.)
[3] Explore by Section
[4] View Section Wise Courses (Bar Graph)
[5] Find Student
[6] Remove Student
[7] View All Students Ranked
[0] Exit

=====
Select action: 4

--- [ SECTION AVERAGE BAR GRAPH ] ---
Enter Section to analyze (Ex: 68_A): 68_A

Generating Bar Graph for Section 68_A (Total Students: 4)

Algorithms(CSE213)           [Avg: 90.8] : #####
Algorithms Lab(CSE214)       [Avg: 83.3] : #####
Engineering Mathematics(MAT211) [Avg: 78.8] : #####
Discrete Mathematics(CSE212) [Avg: 81.0] : #####
Bangladesh Studies(BNS101)   [Avg: 87.8] : #####
```

Find Student Output

```
=====
SMART STUDENT MARKS ORGANIZER
=====
[1] Add Student Record
[2] Top 3 Ranking (CSE Dept.)
[3] Explore by Section
[4] View Section Wise Courses (Bar Graph)
[5] Find Student
[6] Remove Student
[7] View All Students Ranked
[0] Exit
=====
Select action: 5

--- [ SEARCH STUDENT ] ---
Enter Student ID to search: 450

--- [ STUDENT RECORD FOUND ] ---
ID      : 450
Name    : Mohammad
Section : 68_B
-----
Algorithms (CSE213)      : 97
Algorithms Lab (CSE214)  : 96
Engineering Math (MAT211) : 95
Discrete Math (CSE212)   : 94
Bangladesh Studies (BNS101): 85
-----
TOTAL MARKS              : 467
-----
```

Remove Student

```
=====
SMART STUDENT MARKS ORGANIZER
=====
[1] Add Student Record
[2] Top 3 Ranking (CSE Dept.)
[3] Explore by Section
[4] View Section Wise Courses (Bar Graph)
[5] Find Student
[6] Remove Student
[7] View All Students Ranked
[0] Exit
=====
Select action: 6

--- [ REMOVE STUDENT RECORD ] ---
Enter Student ID to remove: 450

Student with ID 450 has been removed successfully!
```

View All Students Rank

```
=====
SMART STUDENT MARKS ORGANIZER
=====
```

- [1] Add Student Record
- [2] Top 3 Ranking (CSE Dept.)
- [3] Explore by Section
- [4] View Section Wise Courses (Bar Graph)
- [5] Find Student
- [6] Remove Student
- [7] View All Students Ranked
- [0] Exit

```
=====
Select action: 7
```

```
--- [ FULL RANKING: ALL CSE STUDENTS ] ---
```

RANK	ID	NAME	SECTION	TOTAL
#1	222	Azmain	68_A	429
#2	457	Delower	68_A	424
#3	548	Tahmid	68_A	417
#4	472	Arif	68_A	416
#5	666	Nadim	68_C	362
#6	444	Faiaz	68_B	340

Chapter 4

Engineering Standards and Mapping

This chapter examines the engineering standards, societal impact, environmental considerations, and program outcome mapping of the Smart Student Marks Organizer project.

4.1 Impact on Society, Environment and Sustainability

4.1.1 Impact on Life

The Smart Student Marks Organizer demonstrates significant positive impact on educational experiences and practical skill development:

Educational Enhancement: By providing a concrete implementation of algorithms (Quick Sort, Binary Search) in a real-world academic context, the project bridges the gap between theoretical algorithm design and practical applications. Students can observe how divide-and-conquer and logarithmic search operate in a meaningful scenario.

Skill Development: The project fosters essential algorithmic thinking skills including:

- Divide-and-conquer strategy implementation
- Recursive algorithm design and analysis
- Search algorithm optimization
- Memory management and pointer operations
- File I/O and data persistence

4.1.2 Impact on Society & Environment

Societal Benefits:

- **Educational Advancement:** Contributes to developing better-prepared software engineers by providing practical algorithm learning tools
- **Academic Management:** Shows how fundamental algorithmic concepts can solve real educational administration problems efficiently
- **Open Knowledge:** Serves as a reference implementation for students learning algorithms and C programming

Environmental Considerations:

- **Minimal Footprint:** As a software-only solution, the environmental impact is negligible
- **Energy Efficiency:** Quick Sort and Binary Search are optimized for performance, reducing computational resource requirements
- **Digital Distribution:** Eliminates paper-based systems it could potentially replace

4.2 Project Management and Teamwork

Our team **Binary Brains**, consisting of **five** members—

1. Sk. Azmain Zunaid (ID: 251-15-222) - **Leader**,
2. Md. Maruf Mia (ID: 251-15-042),
3. Syeda Zawada Farah (ID: 251-15-223),
4. Tahmid Ibne Mofazzol (ID: 251-15-548), and
5. Md. Delower Sarker (ID: 251-15-457)

—worked collaboratively to successfully complete the **Smart Student Marks Organizer** project.

As a team, we understood the importance of dividing tasks while maintaining clear communication and shared goals. We all sat together regularly to discuss the progress, tackle challenges, and ensure that each part of the project was on track.

Team Structure and Role Allocation:

- **Project Leader: Zunaid**, as the team leader, responsible for overall project coordination, task delegation, and timeline management. The leader provided guidance at every stage, performed the initial system design verification, and conducted the final review, correction, and integration of all work—including the code implementation, documentation, and presentation—to ensure quality and coherence.
- **System Implementation Lead: Delower, Tahmid and Zunaid** took the primary responsibility for the hands-on code implementation. This involved writing the core functions, integrating modules, debugging, and verifying the practical functionality of the design.
- **Documentation Team: Farah and Zunaid** was primarily responsible for the comprehensive project report. He diligently compiled the research, wrote detailed descriptions of the methodology, and structured the report as per the required guidelines.
- **Presentation Team: Maruf and Tahmid** was responsible for creating the project presentation slides. He worked to distill the complex project details into a clear, concise, and visually engaging format suitable for demonstration.
- **Testing & Quality Assurance: Maruf** focused on validating the system, designing test cases, and ensuring the robustness of error handling and input validation.
- **Research & Requirements Analysis: Delower and Farah** collaborated on conducting the feasibility study, gap analysis, and compiling reference materials and best practices for the project. Through regular team meetings, we ensured that everyone's ideas and contributions were valued, which allowed us to maintain a balanced approach to complete this project. We collaborated seamlessly to tackle issues, make decisions, and ensure that the project progressed smoothly towards completion.

4.3 Complex Engineering Problem

4.3.1 Mapping of Program Outcome

In this section, provide a mapping of the problem and provided solution with targeted Program Outcomes (PO's).

Table 4.1: Justification of Program Outcomes

PO	Justification of Mapping (Project Perspective)
PO1	The project demonstrates the application of fundamental knowledge of data structures, algorithms, and programming principles. Concepts such as singly linked lists, sorting (Quick Sort), and searching (Binary Search) are implemented to manage and process student data efficiently. Additionally, file handling techniques are used to ensure persistent data storage, reflecting the practical use of core computer science knowledge.
PO2	The project identifies and analyzes the real-world problem of organizing and evaluating student academic records. It breaks the problem into manageable components such as data input validation, storage, retrieval, ranking, and visualization. Appropriate algorithmic solutions are selected (e.g., sorting for ranking, searching for retrieval), ensuring correctness and efficiency in handling different user queries.

4.3.2 Complex Problem Solving

Table 4.2: Mapping with complex problem solving.

EP1 Dept of Knowledge	EP2 Range of Conflicting Requirements	EP3 Depth of Analysis	EP4 Familiarity of Issues	EP5 Extent of Applicable Codes	EP6 Extent Of Stakeholder Involvement	EP7 Inter- dependence
✓		✓				

EP1: The project requires the application of in-depth knowledge of data structures (linked lists), algorithm design (Quick Sort, Binary Search), and file handling mechanisms. It integrates multiple core concepts to develop a complete system for managing and analyzing student records. The combination of dynamic memory management, efficient data organization, and algorithmic processing reflects the need for substantial foundational and applied knowledge.

EP3: The project involves detailed analysis in selecting appropriate algorithms and designing efficient workflows. For example, Quick Sort is used to rank students based on total marks, ensuring efficient performance for larger datasets, while Binary Search enables faster retrieval of student records. Additionally, the system handles validation, edge cases (e.g., duplicate IDs, empty database), and structured data processing, demonstrating a deeper level of analytical thinking in problem-solving.

Chapter 5

Conclusion

5.1 Summary

The Smart Student Marks Organizer successfully demonstrates the practical application of fundamental algorithms in solving real-world academic record management problems. The project implements a complete student management solution using linked lists, Quick Sort for ranking, and Binary Search for efficient lookup, all developed in ANSI C.

Key Achievements:

- **Algorithm Implementation:** Successfully implemented Quick Sort ($O(n \log n)$) for top 3 ranking and Binary Search ($O(\log n)$) for student lookup
- **Efficient Data Management:** Sorted linked list insertion maintaining ascending ID order for optimal search performance
- **Comprehensive Student Management:** Full CRUD operations on student records with persistent file storage
- **Section-Wise Analysis:** Section filtering and bar graph visualization of course averages
- **Robust Error Handling:** Comprehensive input validation and error management throughout the system
- **Proper Memory Management:** Correct allocation and deallocation of dynamic memory
- **Educational Value:** Created a practical learning tool demonstrating algorithm design principles

The system successfully meets all specified objectives, providing both functional student record management capabilities and valuable educational insights into algorithm implementation.

5.2 Limitations

Despite achieving its primary objectives, the system has several limitations:

1. **Functionality Gaps:**
 - No update/modify student record feature
 - No batch import/export functionality
 - Basic console interface without graphical elements
2. **Scalability Constraints:**
 - Quick Sort worst-case $O(n^2)$ for already sorted data
 - Binary Search requires conversion of linked list to array ($O(n)$ overhead)
 - Linear search for section filtering ($O(n)$ per query)
3. **Algorithm Limitations:**
 - Quick Sort pivot selection as first element (suboptimal for sorted data)
 - No caching mechanism for frequently accessed data

5.3 Future Work

The Smart Student Marks Organizer provides a solid foundation for enhancements:

1. Algorithm Improvements:

- Implement randomized pivot selection for Quick Sort to avoid worst-case $O(n^2)$
- Use Balanced Binary Search Tree (AVL/Red-Black) for $O(\log n)$ search without array conversion
- Implement Merge Sort for guaranteed $O(n \log n)$ performance

2. Enhanced Functionality:

- Add student record update/modify feature
- Implement batch import/export (CSV/Excel)
- Add graphical user interface (GUI) using GTK or Qt
- Implement course-wise ranking and statistics

3. Advanced Features:

- Add grade calculation (A+, A, B+, etc.)
- Implement transcript generation
- Add semester-wise performance tracking
- Implement predictive analytics for student performance

4. Security Enhancements:

- Add role-based access control (Admin/Faculty/Student)
- Implement encrypted database storage
- Add audit logging for all operations

References

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- [2] Hoare, C. A. R. (1961). "Quicksort". *The Computer Journal*, 5(1), 10-16.
- [3] Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.
- [4] Kanetkar, Y. (2003). *Data Structures Through C* (2nd ed.). BPB Publications.
- [5] International Organization for Standardization. (2018). *ISO/IEC 9899:2018 — Programming Languages — C*. ISO.